

AegisNEO

Official Product Requirements Document (PRD)

System Architecture & UI/UX Guidelines for AI Implementation

Target Platform: Vercel Serverless Functions

1 Project Overview

AegisNEO is a full-stack web application designed to track and display Near-Earth Objects (Asteroids) using data derived from the NASA Near-Earth Object Dataset. The primary purpose of this project is to demonstrate cloud deployment capabilities (via Vercel), backend API routing with FastAPI, and dynamic DOM manipulation using vanilla JavaScript.

CRITICAL CONTEXT NOTE FOR AI (CLAUDE): The user has been provided a “Car Database” API by their professor as a starter sample. **This Car API serves as the strict ARCHITECTURAL template for this project.** You must follow the exact same logic patterns (e.g., using vanilla JS `fetch`, and using JS template literals to inject HTML, replacing the old `displayCars()` logic).

However, while the logic must mirror the starter code, you have absolute freedom to completely overhaul the HTML layout and CSS styling to achieve a massive visual upgrade.

Crucial Dataset Requirement: The application must successfully handle a JSON dataset containing upwards of 30,000 to 90,000 items to prove the application scales properly and that the search endpoints filter data efficiently.

2 Strict Technical Constraints (READ CAREFULLY)

AI Agent Directive: You are to act as a Senior Full-Stack Developer. You must strictly adhere to the following technological boundaries. Failure to follow these constraints will result in a failing grade for the user.

- **NO FRONTEND FRAMEWORKS:** Do not use React, Vue, Svelte, Angular, or Next.js.
- **Frontend Stack:** Strictly Vanilla HTML5 (`index.html`), CSS3 (`style.css`), and JavaScript ES6 (`app.js`).
- **Backend Stack:** Python 3.9+, FastAPI, Uvicorn.
- **Deployment:** Vercel (Frontend deployed as static files; Backend deployed via `api/index.py` or `index.py` as a serverless function).

3 Backend API Architecture (`index.py`)

The backend will serve a static JSON file containing the asteroid data.

3.1 Data Structure (`asteroids.json`)

The dataset contains 30,000+ objects. Each object includes:

- `neo_reference_id`: String
- `name`: String (e.g., “433 Eros”)

- `estimated_diameter_km`: Float
- `is_potentially_hazardous`: Boolean
- `close_approach_date`: String (e.g., "2024-11-20")
- `relative_velocity_km_h`: Float
- `miss_distance_km`: Float

3.2 Endpoints & Logic

1. GET `/asteroids`

- Must accept an optional `?search=` query parameter.
- The search logic must evaluate the `name` and `neo_reference_id` fields (case-insensitive).
- **Performance Mandate:** The endpoint *must* slice the returned array to a maximum of 100 items to prevent the vanilla JS frontend from crashing when rendering the DOM.

2. GET `/asteroids/{neo_reference_id}`

- Returns a single object matching the reference ID.

3.3 Middleware Requirement

FastAPI must include **CORS (Cross-Origin Resource Sharing)** middleware allowing `allow_origins=["*"]`. This is mandatory for the local frontend to successfully fetch from the local (or deployed) back-end.

4 Frontend UI/UX Guidelines (Pro Max Design)

The application must feature a highly aesthetic, premium "Pro Max" telemetry UI/sci-fi theme. It should look like a production-ready application, completely distinct from generic AI-generated interfaces.

4.1 Design System

- **Backgrounds:** Use `assets/hero-bg.jpg` as a fixed, deep cosmos background.
- **Components (Glassmorphism):** Cards and UI elements should have a translucent background (e.g., deep void black with `backdrop-filter: blur(12px)`) and a subtle 1px cyan or amber border depending on hazard status.
- **Typography:** Use 'Space Grotesk' or 'JetBrains Mono' for telemetry data headers and 'Inter' for standard body text. Maintain high contrast (off-white/cyan text on dark backgrounds).

4.2 JavaScript Integration (`app.js`)

- Fetch data from the API endpoint using vanilla `fetch()`.
- Dynamically inject data into the DOM using template literals (`` ``), transforming raw numeric metrics into human-readable telemetry.

- **Loading State:** Display a sleek CSS spinner or radar sweep animation while the fetch request is pending.
- **Empty State:** Display a stylized "No objects detected in this sector" message if the search yields zero results.
- **Asset Mapping:** Inside the template literals, use the UI icons provided in the assets folder (e.g., `assets/icon-warning.svg`, `assets/icon-safe.svg`) based on the `is_potentially_hazardous` boolean.

5 Required Deliverables to Generate

Claude must output the complete code for the following files to ensure a seamless Vercel deployment:

- `index.py` (FastAPI Backend)
- `index.html` (Structure)
- `style.css` (Premium Theming)
- `app.js` (Logic and DOM manipulation)
- `requirements.txt` (Python dependencies: `fastapi`, `uvicorn`)
- `vercel.json` (Serverless configuration routing `/api/*` to `index.py`)